

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 7: PARALELIZAÇÃO UTILIZANDO “#pragma omp task”

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 28/08/2026.

1. EXPERIMENTO

Implemente um programa em C que cria uma lista encadeada com nós, cada um, contendo o nome de um arquivo fictício. Dentro de uma região paralela, percorra a lista e crie uma tarefa com “#pragma omp task” para processar cada nó. Cada tarefa deve imprimir o nome do arquivo e o identificador da thread que a executou com o “omp_get_thread_num”. Após executar o programa, reflita: todos os nós foram processados? Algum foi processado mais de uma vez ou ignorado? O comportamento muda entre execuções? Como garantir que cada nó seja processado uma única vez e por apenas uma tarefa?

2. CONCEITOS RELEVANTES

- **“pragma omp task”:** Em vez de dividir iterações de um laço rigidamente entre threads (como o “#pragma omp for faz”), o omp task permite definir um bloco de código que qualquer thread disponível no “pool” poderá pegar e executar de forma assíncrona. Uma thread encontra a diretiva e apenas *empacota* a tarefa na fila. A execução real pode ocorrer imediatamente, depois, ou por qualquer outra thread da região paralela.

- **“pragma omp single”:** garante que o bloco de código associado seja executado por apenas uma thread do pool, enquanto todas as outras threads da região paralela aguardam. A escolha de qual thread vai executar esse bloco é feita pelo próprio ambiente de execução do OpenMP (geralmente a primeira thread que chegar naquele ponto).
- **“taskwait” e “nowait”:** O “taskwait” força uma pausa, bloqueia a thread atual até que as tarefas que ela criou finalizem. Já o “nowait” remove uma pausa. Diz para as threads não esperarem umas pelas outras no final de uma diretiva.

3. MÁQUINA UTILIZADA

Para a execução deste experimento, estou utilizando uma máquina equipada com um processador AMD Ryzen 7 5700X (8 núcleos físicos e 16 threads), que é o componente central para a avaliação das diretivas OpenMP. O sistema dispõe também de 32 GB de memória RAM e um SSD de 1TB, garantindo um ambiente isolado de gargalos de alocação de memória ou paginação pelo sistema operacional, o que assegura a precisão das métricas de tempo.

4. CÓDIGO

Para o experimento, inicialmente o código representa o uso das tasks sem o controle das “threads”. O programa (Figura 1) demonstra a estrutura base do experimento (alocação de nós e uso de “#pragma omp task”), porém serve como exemplo de paralelização incorreta/ingênua ao omitir o “#pragma omp single”, todas as threads percorrem a lista e enfileiram tarefas duplicadas.

Figura 1: Código completo sem controle das “threads”

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

typedef struct No {
```

```c
    char nome_arquivo[80];
    struct No *proximo;
} No;

No *criar_no(const char *nome_arquivo, No *proximo) {
    No *novo_no = malloc(sizeof(No));

    if (novo_no == NULL) {
        return NULL;
    }

    snprintf(novo_no->nome_arquivo, sizeof(novo_no->nome_arquivo), "%s", nome_arquivo);
    novo_no->proximo = proximo;
    return novo_no;
}

int inserir_no_inicio(No **lista, const char *nome_arquivo) {
    No *novo_no = criar_no(nome_arquivo, *lista);

    if (novo_no == NULL) {
        return 0;
    }

    *lista = novo_no;
    return 1;
}


void liberar_lista(No *lista) {
    while (lista != NULL) {
        No *proximo = lista->proximo;
        free(lista);
        lista = proximo;
    }
}

int main(void) {
```

```
  const char *musicas[] = {
    "1_Atrasadinha.mp3",
    "2_Pirata_e_Tesouro.mp3",
    "3_Eu_Juro.mp3",
    "4_Sinto_Sua_Falta.mp3",
    "5_Pra_Voce_Acreditar.mp3",
    "6_Climatizar.mp3",
    "7_Me_Perdoa.mp3",
    "8_Casa_do_Amor.mp3",
    "9_Tristinha.mp3",
    "10_To_Isso_Aqui_Pra_Te_Beijar.mp3"
  };
  const size_t quantidade = sizeof(musicas) / sizeof(musicas[0]);
  No *lista = NULL;

  for (size_t i = 0; i < quantidade; i++) {
    if (!inserir_no_inicio(&lista, musicas[i])) {
      fprintf(stderr, "Erro ao alocar memoria para a lista.\n");
      liberar_lista(lista);
      return 0;
    }
  }



  #pragma omp parallel
  {
    No *no = lista;

    while (no != NULL) {
      No *no_atual = no;

      #pragma omp task
      printf("Arquivo: %s | Thread: %d\n",  no_atual->nome_arquivo,
omp_get_thread_num());

      no = no->proximo;
    }
```

```
    }

    liberar_lista(lista);

    return 0;
}
```

Após os testes, o código foi corrigido com o uso do single, sendo a versão final do loop representada na figura abaixo (Figura 2).

Figura 2: Trecho de loop ajustado

```
#pragma omp parallel
{
    #pragma omp single
    {
        No *no = lista;
        printf("Thread %d percorrendo a lista encadeada.\n", omp_get_thread_num());

        while (no != NULL) {
            No *no_atual = no;

            // o omp task por padrão faz o comportamento de firstprivate,
                mas é uma boa prática explicitar.
            // isso garante que cada tarefa tenha sua própria cópia da variável no_atual,
                evitando problemas de concorrência.
            #pragma omp task firstprivate(no_atual)
            printf("Arquivo: %s | Thread: %d\n",
                  no_atual->nome_arquivo,
                  omp_get_thread_num());

            no = no->proximo;
        }

        // há um taskwait a seguir... não necessário, apenas garantia.
        // Se houvesse um nowait, o programa aguardaria a conclusão de todas as tarefas
```

```
            antes de prosseguir.
        #pragma omp taskwait
    }
}
```

5. RESULTADOS

Abaixo, é possível visualizar o resultado da compilação dos códigos antes e depois da alteração.

Figura 3: Saída do programa antes da alteração (3 threads)

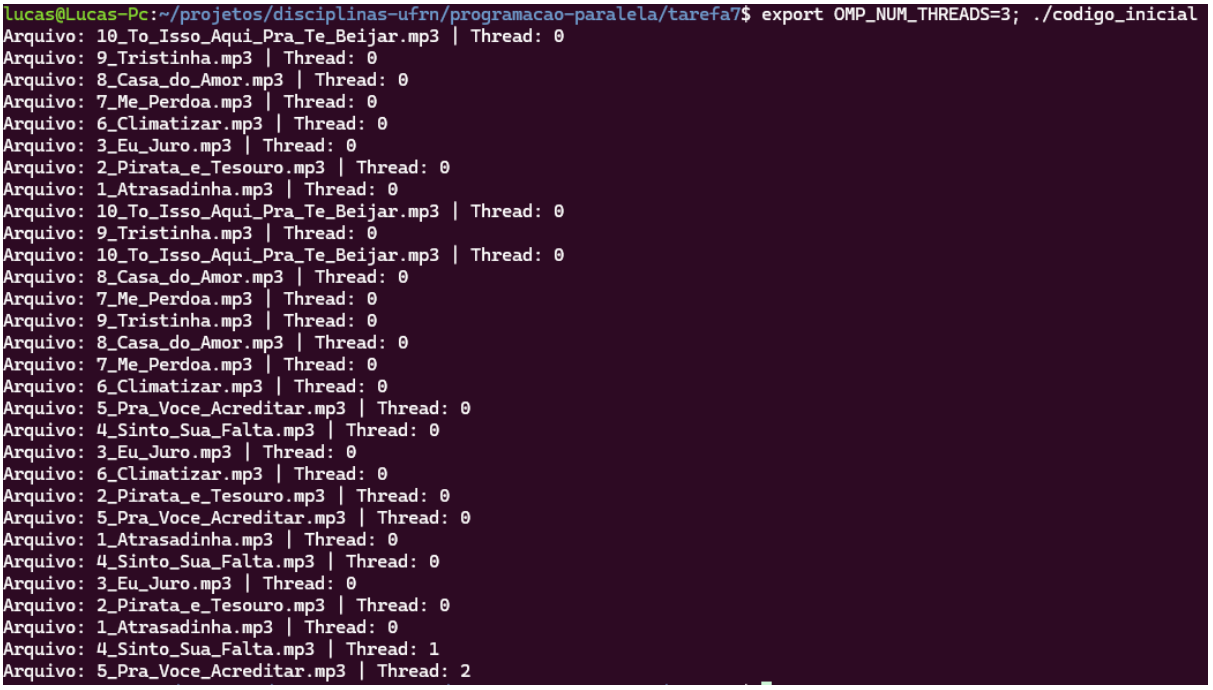


Figura 4: Saída do programa antes da alteração (4 threads)

```
lucas@Lucas-Pc:~/projetos/disciplinas-ufrn/programacao-paralela/tarefa7$ export OMP_NUM_THREADS=4; ./codigo_inicial
Arquivo: 10_To_Isso_Aqui_Pra_Te_Beijar.mp3 | Thread: 0
Arquivo: 9_Tristinha.mp3 | Thread: 0
Arquivo: 8_Casa_do_Amor.mp3 | Thread: 0
Arquivo: 7_Me_Perdoa.mp3 | Thread: 0
Arquivo: 6_Climatizar.mp3 | Thread: 0
Arquivo: 5_Pra_Voce_Acreditar.mp3 | Thread: 0
Arquivo: 4_Sinto_Sua_Falta.mp3 | Thread: 0
Arquivo: 3_Eu_Juro.mp3 | Thread: 0
Arquivo: 2_Pirata_e_Tesouro.mp3 | Thread: 0
Arquivo: 1_Atrasadinha.mp3 | Thread: 0
Arquivo: 10_To_Isso_Aqui_Pra_Te_Beijar.mp3 | Thread: 0
Arquivo: 8_Casa_do_Amor.mp3 | Thread: 3
Arquivo: 7_Me_Perdoa.mp3 | Thread: 3
Arquivo: 10_To_Isso_Aqui_Pra_Te_Beijar.mp3 | Thread: 3
Arquivo: 9_Tristinha.mp3 | Thread: 3
Arquivo: 7_Me_Perdoa.mp3 | Thread: 3
Arquivo: 6_Climatizar.mp3 | Thread: 3
Arquivo: 8_Casa_do_Amor.mp3 | Thread: 1
Arquivo: 3_Eu_Juro.mp3 | Thread: 1
Arquivo: 2_Pirata_e_Tesouro.mp3 | Thread: 1
Arquivo: 1_Atrasadinha.mp3 | Thread: 1
Arquivo: 10_To_Isso_Aqui_Pra_Te_Beijar.mp3 | Thread: 1
Arquivo: 9_Tristinha.mp3 | Thread: 1
Arquivo: 8_Casa_do_Amor.mp3 | Thread: 1
Arquivo: 7_Me_Perdoa.mp3 | Thread: 1
Arquivo: 6_Climatizar.mp3 | Thread: 1
Arquivo: 5_Pra_Voce_Acreditar.mp3 | Thread: 1
Arquivo: 4_Sinto_Sua_Falta.mp3 | Thread: 1
Arquivo: 3_Eu_Juro.mp3 | Thread: 1
Arquivo: 2_Pirata_e_Tesouro.mp3 | Thread: 1
Arquivo: 1_Atrasadinha.mp3 | Thread: 1
Arquivo: 6_Climatizar.mp3 | Thread: 1
Arquivo: 5_Pra_Voce_Acreditar.mp3 | Thread: 1
Arquivo: 4_Sinto_Sua_Falta.mp3 | Thread: 1
Arquivo: 3_Eu_Juro.mp3 | Thread: 1
Arquivo: 2_Pirata_e_Tesouro.mp3 | Thread: 1
Arquivo: 1_Atrasadinha.mp3 | Thread: 1
Arquivo: 9_Tristinha.mp3 | Thread: 0
```

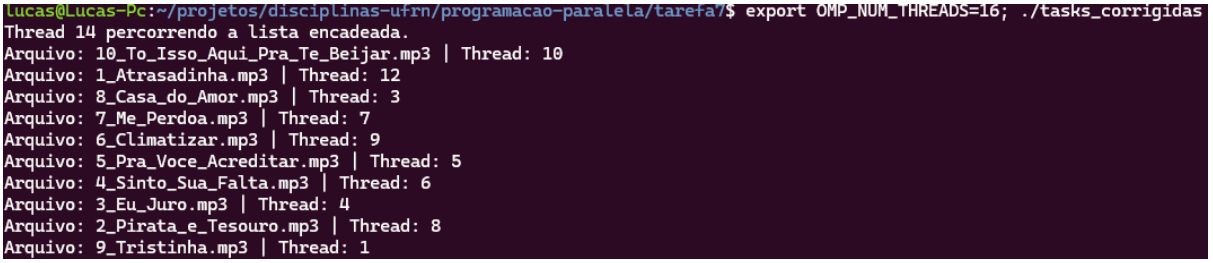
Figura 5: Saída do programa após a alteração (3 threads)

```
lucas@Lucas-Pc:~/projetos/disciplinas-ufrn/programacao-paralela/tarefa7$ export OMP_NUM_THREADS=3; ./tasks_corrigidas
Thread 1 percorrendo a lista encadeada.
Arquivo: 10_To_Isso_Aqui_Pra_Te_Beijar.mp3 | Thread: 0
Arquivo: 8_Casa_do_Amor.mp3 | Thread: 0
Arquivo: 7_Me_Perdoa.mp3 | Thread: 0
Arquivo: 6_Climatizar.mp3 | Thread: 0
Arquivo: 5_Pra_Voce_Acreditar.mp3 | Thread: 0
Arquivo: 9_Tristinha.mp3 | Thread: 2
Arquivo: 4_Sinto_Sua_Falta.mp3 | Thread: 0
Arquivo: 2_Pirata_e_Tesouro.mp3 | Thread: 0
Arquivo: 3_Eu_Juro.mp3 | Thread: 2
Arquivo: 1_Atrasadinha.mp3 | Thread: 1
```

Figura 6: Saída do programa após a alteração (4 threads)

```
lucas@Lucas-Pc:~/projetos/disciplinas-ufrn/programacao-paralela/tarefa7$ export OMP_NUM_THREADS=4; ./tasks_corrigidas
Thread 2 percorrendo a lista encadeada.
Arquivo: 10_To_Isso_Aqui_Pra_Te_Beijar.mp3 | Thread: 0
Arquivo: 7_Me_Perdoa.mp3 | Thread: 0
Arquivo: 6_Climatizar.mp3 | Thread: 0
Arquivo: 5_Pra_Voce_Acreditar.mp3 | Thread: 0
Arquivo: 4_Sinto_Sua_Falta.mp3 | Thread: 0
Arquivo: 3_Eu_Juro.mp3 | Thread: 0
Arquivo: 2_Pirata_e_Tesouro.mp3 | Thread: 0
Arquivo: 1_Atrasadinha.mp3 | Thread: 2
Arquivo: 8_Casa_do_Amor.mp3 | Thread: 1
Arquivo: 9_Tristinha.mp3 | Thread: 3
```

Figura 6: Saída do programa após a alteração (16 threads)



A verificação do comportamento do programa foi realizada em duas etapas, sendo a primeira sem a restrição de escopo de criação de tarefas e, em seguida, aplicando a diretiva “#pragma omp single”.

Na primeira execução, o programa foi configurado com “OMP_NUM_THREADS=3”. Como demonstrado na primeira imagem, ocorreu uma redundância maciça no processamento dos nós. Como o laço “while” não estava contido em uma região de thread única, todas as 3 threads iniciadas pelo “#pragma omp parallel” percorreram a lista encadeada de forma independente. Com isso, o mesmo nó, como os arquivos “10_To_Isso_Aqui_Pra_Te_Beijar.mp3” e “9_Tristinha.mp3”, foi lido e colocado na fila de tarefas três vezes distintas, uma por cada thread que iterava sobre a lista. O terminal exibe múltiplos arquivos repetidos sendo impressos pela “Thread 0”, seguidos por execuções residuais das “Threads 1 e 2” no final da saída. Esse comportamento confirma que múltiplos nós foram processados mais de uma vez, violando a regra de execução única.

Na segunda execução, apresentada nas imagens seguintes (Figuras 5 e 6), o programa foi executado com “OMP_NUM_THREADS=4” após encapsular a iteração da lista dentro da diretiva “#pragma omp single”. A mensagem inicial indicando que a “Thread 2” estava percorrendo a lista encadeada deixa claro que apenas uma thread assumiu o papel de percorrer a estrutura de dados e enfileirar as tarefas com “#pragma omp task”. A lista continha exatamente 10 arquivos e o terminal registra exatas 10 saídas impressas, confirmando que nenhum nó foi ignorado e nenhum foi processado em duplicidade. Embora a “Thread 2” tenha gerado as tarefas de forma sequencial, a execução foi distribuída entre as threads disponíveis no pool, onde a “Thread 0” processou a maioria das faixas, enquanto as Threads “1, 2 e 3” assumiram os arquivos restantes de forma assíncrona.

A comparação entre as duas imagens evidencia que a diretiva #pragma omp single é indispensável ao utilizar tarefas do OpenMP em estruturas dinâmicas como listas encadeadas, pois garante a separação clara entre a fase sequencial de varredura e a fase paralela de processamento.

6. CONSIDERAÇÕES FINAIS

O experimento permitiu compreender na prática o funcionamento e a aplicação do modelo de tarefas do OpenMP (“#pragma omp task”) no contexto de estruturas de dados dinâmicas e não contíguas, como a lista encadeada. Diferentemente de laços de repetição tradicionais com tamanho fixo, em que a divisão do trabalho pode ser feita por fatiamento de índices via “#pragma omp for”, estruturas encadeadas exigem uma abordagem baseada em tarefas assíncronas devido à natureza estritamente sequencial do percurso por ponteiros.

Foi demonstrado que a simples declaração de uma região paralela sem o devido controle de escopo gera concorrência indesejada e redundância severa, fazendo com que todas as threads percorram a lista e dupliquem o processamento dos nós. A aplicação da diretiva `#pragma omp single` provou ser a solução ideal para esse padrão de projeto, permitindo que uma única thread atue na varredura sequencial da estrutura e no enfileiramento ultrarrápido dos trabalhos, enquanto o restante do *pool* de threads atua de forma cooperativa na execução paralela das tarefas.

Por fim, o experimento evidencia o potencial do OpenMP Tasking para acelerar algoritmos que operam sobre estruturas dinâmicas, unindo simplicidade de implementação e eficiência no gerenciamento de recursos concorrentes.